The Exponential Time Hypothesis Proof Checks, and Its Algorithm Is Brute Force on Every Formula That Could Exist

ZKSF (Zero Kelvin Simulation Foundry) · 28 October 2026 · Version 1.0 · Replication data at github.com/official-dvl/zksf-eth-checker

Abstract

On 26 October 2026 a preprint and a machine-checked Lean proof claimed a deterministic algorithm that counts the satisfying assignments of any k-CNF formula in time 2^{O(n log log n / √log n)}, which refutes the Exponential Time Hypothesis. We re-ran the published check from the pinned commit in an isolated container, three times. Lean's kernel accepts the proof with only the three standard axioms, and so does nanoda, comparator's independently written second kernel, which the published configuration leaves off. Every input to the check is byte-identical to its published source. We then read where the algorithm's subexponential construction is actually used. The paper's own rule in Proposition 6.4 reads "If s < 2, we use exhaustive search", and by that rule every formula with at most 2^521,283 variables, a number with 156,922 digits, is solved by exhaustive search. The formally verified program uses its construction only above 2^639,999. Both bounds are checked in Lean against the proof's own definitions. Even with every hidden constant set to 1, the claimed time bound is no better than brute force for any n up to 2^65,536. The result is correct as mathematics and says nothing about solving SAT on any instance that can be written down. All scripts, logs and hashes needed to repeat each step are published.

1. The claim

- The source is `github.com/openai/math` at commit `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a` (26 Oct 2026), with the paper `preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf` and the Lean development in `lean/`.
- Theorem 1.1 states that for each fixed k ≥ 2, exact #k-SAT runs in (n+m)^{O_k(1)} · 2^{O_k(n log log n / √log n)} bit operations and polynomial space. The Lean statement proves the exponent n (log log n)^2 / √log n.
- The formal statement is theorem `exact_count_and_sat` in `ComparatorChallenges/SubexponentialSAT.lean`. For each k, one fixed multi-tape machine, given the canonical encoding of a formula, writes the exact count and the satisfiability bit within C (L+1)^d · 2^{A n (log log(n+16))^2 / √log(n+16)} steps and S (L+1)^e visited cells.
- **ZKSF's audit of this statement (the whole 7.8 KB file) finds it faithful.** The machine is a standard deterministic multi-tape Turing machine over the alphabet {blank, 0, 1} with finitely many states and tapes. Each step performs one action, a move or a write, on one tape, so time is counted honestly. There is one machine for each k, fixed before n and F are chosen, which makes it a uniform algorithm with no per-size circuits. The input holds only the formula, with n in unary and then the clauses as sorted lists of literal codes. The machine must halt with the answer bit as its final state, leave the exact count in binary on one tape (checked bit for bit) and the answer bit on another, for every n and every formula of width at most k, within the time bound, using polynomially many cells at every step of the run. Two features cost at most polynomial factors, which the bound absorbs.

These are one tape action per step and literal codes written in unary. For k = 3 the input has polynomially many bits in n, so the statement gives 3-SAT in 2^{o(n)} time, the negation of ETH.

- The check is Lean FRO's `comparator`. The solution module must prove the challenge's statement using only `propext`, `Quot.sound` and `Classical.choice`. The published configuration leaves comparator's second, independently written kernel, nanoda, switched off.

2. Re-running the check

Setup. We built a Docker image from pinned sources with Lean 4.34.1 (5045d0056413), comparator and lean4export v4.34.0, landrun (ec104f6edc1e) and nanoda (37cb87b7e6cd). Mathlib is pinned at d13f23b723b8 in both the lakefile and the manifest, and we fetched it from the official cache with `lake exe cache get`. We never ran `lake update`, which would move dependencies pinned to `main`.

Isolation was a Docker container with no network, as a non-root user. The checker's additional file-level layer (landrun) needs Linux 7.2+, so on our 5.15 kernel we added the integrity checks below.

Run 1, the published configuration. It passed with "Lean default kernel accepts the solution" and "Your solution is okay!", exit 0, after 12,014 build jobs and 1 h 30 min on 18 threads.

Integrity. Nothing outside the work volume changed (`docker diff`). The challenge file, its configuration, the lakefile, the manifest and the toolchain file have GitHub's git blob hashes at the pinned commit. All 8,926 compiled Mathlib and dependency files are byte-identical to a fresh download of the official cache, with 0 different and 0 missing.

Run 2, nanoda switched on. The challenge's build outputs were set aside so the challenge was rebuilt from source. The rebuilt files are byte-identical to the originals, and no file of the solution was rebuilt. Lean's kernel accepted again. nanoda reached no verdict because it aborted after 80 s on a stack overflow in one of its checking threads (exit 134), which comparator reports as "nanoda kernel rejected the solution". The cause is nanoda's fixed 16 MiB thread stack.

Run 3, nanoda with a 1 GiB thread stack. That constant is the only change, built offline from the same source. Both kernels accept, with "nanoda kernel accepts the solution", "Lean default kernel accepts the solution" and "Your solution is okay!", exit 0. The two checks together took 31 minutes, and again no file of the solution was rebuilt.

3. Where the algorithm is used

The paper's own rule. The proof of Proposition 6.4 (pp. 43 to 44) handles $n < 20$ by exhaustive search. It then defines $\lambda = \lceil \log_2 n \rceil$, $\nu = \lceil \log_2 \lambda \rceil$ and $s = \lfloor \sqrt{\lambda} / \nu^2 \rfloor$ and continues "If $s < 2$, we use exhaustive search; $\sqrt{\lambda}/\nu^2$ tends to infinity, so there are only finitely many such sizes", later adding "the finite exhaustive-search cases are absorbed in the constants". It does not say how many. Computing s exactly, $s \geq 2$ only for λ between 521,284 and 524,288 and for $\lambda \geq 640{,}000$. So the paper's algorithm is exhaustive search for **every formula with $n \leq 2^{521{,}283}$ variables**, and again for $2^{524{,}288} < n \leq 2^{639{,}999}$.

The verified program. The Lean definition `sparsifySchedule` is the paper's s. The program's threshold is `max 20 (max Nleaf Ns)`, with Ns any number past which the schedule stays at 2 or more (`NativeLargeEntry.lean`, lines 48 to 50). No value is written anywhere. Below the threshold the program answers from a finite table fixed before the input (`FiniteFallback.lean`, `noncomputable`), in at most 28,700 $(L+1)^2$ steps.

Checked in Lean. The file `cutoff_check.lean` compiles against the proof's own definitions with standard axioms only.

- `sparsifySchedule (2^521283) = 1` and `sparsifySchedule (2^521284) = 2` mark where the paper's s first reaches 2.
- `sparsifySchedule (2^639999) = 1` and `sparsifySchedule (2^640000) = 2`.
- `zksf_Ns_gt` shows that every Ns the proof's construction can take exceeds 2^639,999. So the verified program never uses its subexponential construction on a formula with at most 2^639,999 variables (192,659 digits).

The bound itself. Independently of these parameters, the exponent $n (\log \log n)^2 / \sqrt{\log n}$ is below n only when $(\log_2 x)^4 < x$ with $x = \log_2 n$, and this fails for every n from 2^5 to 2^65,536 (a 19,729-digit number). So even with every hidden constant equal to 1, the claimed running time is no better than brute force's 2^n anywhere below that size.

What this means. ETH is a statement about growth rates, and a 2^{o(n)} algorithm refutes it however late the speedup begins, so nothing here contradicts the result. But the sizes involved exceed the roughly 10^80 atoms in the observable universe by more than 156,000 orders of magnitude. The theorem, the paper's algorithm and the verified program all say nothing about solving SAT faster on any formula that can be written down.

4. Quantum computers

- On every real formula the algorithm is exhaustive search, the setting where Grover search finds a solution in about 2^{n/2} steps instead of 2^n.
- Exact counting behaves differently. It includes the parity of the number of solutions, and parity of N bits needs N/2 quantum queries (Beals et al., 2001), so generic quantum search cannot speed up exact counting by more than a constant.
- **Measured on real engines.** The formula (x0 ∨ ¬x1)(x1 ∨ ¬x2)(x2 ∨ ¬x0) has 2 solutions among 8 assignments by exhaustive search. One Grover iteration, with the oracle built from the clauses alone on 6 qubits, puts all probability on them. Every result carries a ZKSF certificate that zcc-verify checks without an account.

Engine	Shots	On the 2 solutions	Certificate	Hardware fidelity
exact.cpu	1,000	100%	818bfc6914fa4377 (ZCC-v0.1, exact)	exact
exact.gpu	1,000	100%	f385b7b3d59f4454 (ZCC-v0.1, exact)	exact
exact.tpu	1,000	100%	7dbcaa36f63c4b9c (ZCC-v0.1, exact)	exact
IQM Garnet	560	44.1%	e2b1fe6a617649ec (ZHF-v0.1)	0.437
AQT IBEX	100	44%	c752e4db92bb418a (ZHF-v0.1)	0.416
Rigetti Cepheus-1-108Q	1,000	26.2%	3b7a1267e203441a (ZHF-v0.1)	0.252

Random guessing gives 25%. On Rigetti's native gates the circuit compiles to about 73 to 81 two-qubit gates. Today's hardware keeps part of the signal (IQM Garnet, AQT) or almost none of it (Rigetti). IonQ Forte and IQM Emerald were offline at the time.

5. Replication

The accompanying repository holds the Dockerfiles, every command, the cutoff script, the Lean file and our logs with hashes. A full check takes about 2 hours on 18 threads, about 20 GB of disk and 24 GB of memory.

References

- openai/math, commit 7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a, and "The Exponential Time Hypothesis is False" (25 Oct 2026).
- Lean FRO, comparator v4.34.0, and nanoda (ammkrn/nanoda_lib).
- R. Beals, H. Buhrman, R. Cleve, M. Mosca, R. de Wolf, "Quantum lower bounds by polynomials", J. ACM 48(4), 2001.
- R. Impagliazzo, R. Paturi, "On the complexity of k-SAT", JCSS 62(2), 2001.